

OFFZONE
2026

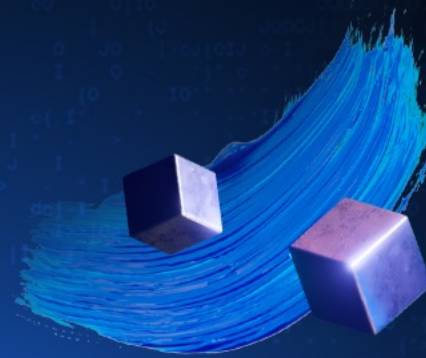
Building AI Pentester: our experience

Sergey Kolesnikov

CICADA8



Not a guide, just experience



Sergey Kolesnikov

Director of Products and Services
(CPO/CTO) at CICADA8

Ex BI.ZONE, ex ByteDance

OSCP, OSWE, CDP, CDE, CREST ...



@tr3harder

Not in slides: ML/DL



Agenda

01 First steps in development

02 Challenges and problems

03 Building the architecture

04 System components

05 Results and takeaways



Timeline

2025

Development start

AI module in ETM solution for
vulnerability verification

2026

Testing new hypotheses

Evaluating open-source libraries and
benchmarking ready-made solutions

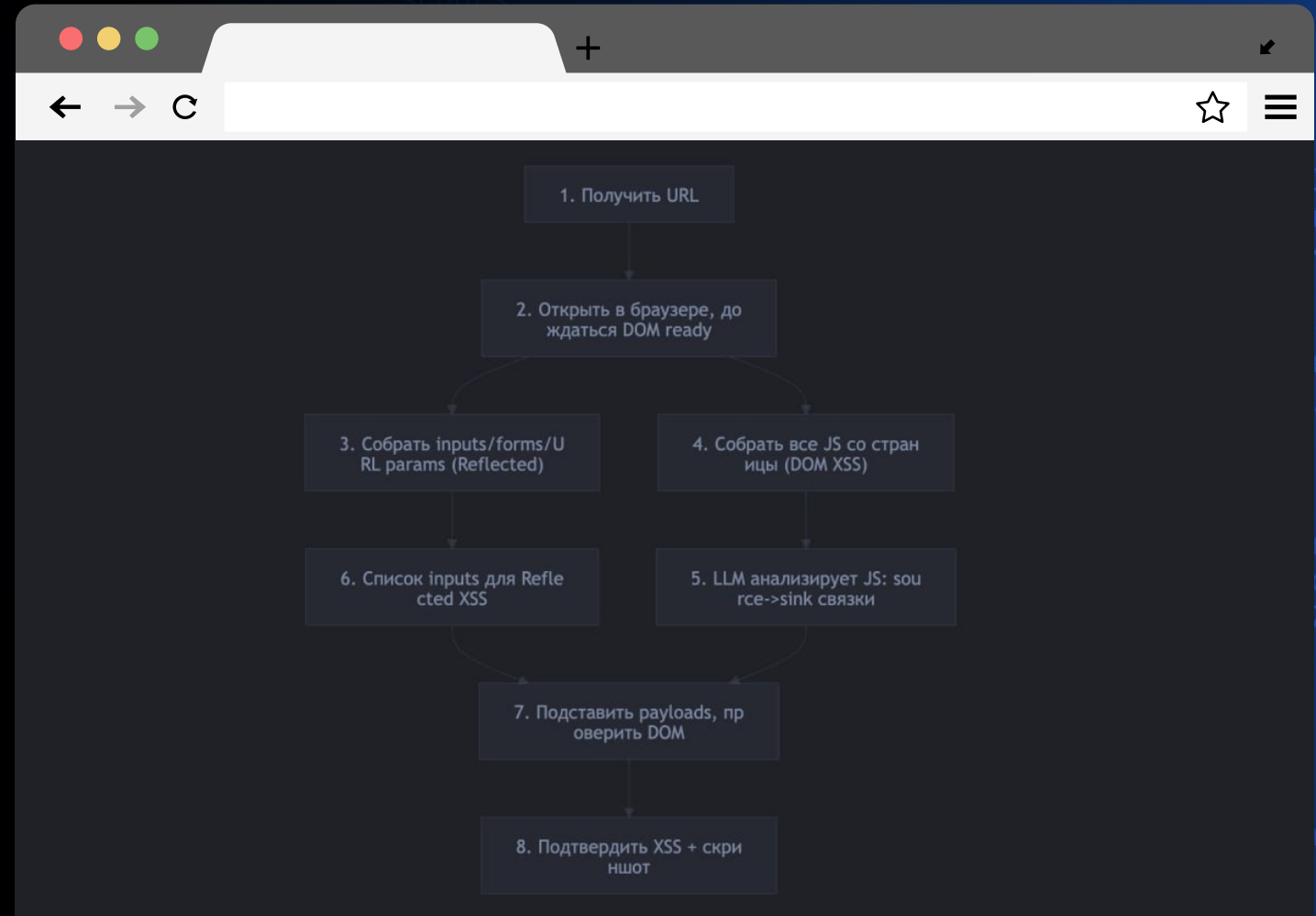
Now

In-house harness

Our own approach, technical
enrichment

First steps

- Started developing an AI module for ETM in 2025.
Goal: continuous perimeter testing
- Built deterministic graphs for different vulnerability classes
- Did a lot of prompt engineering
- Benchmarks on public tasks and labs (XBOW, PortSwigger, HackTheBox)



First steps

First results arrived — and with them, conclusions and problems:

- Architecture depends on the LLM
- Scaling in the current paradigm is hard

Reflected XSS Confirmed

Detail	Information
Target URL	[REDACTED]
Injection Vector	GET parameter reason (reflected in HTML via innerHTML)
Reflection Context	HTML text – the value is inserted directly into <div id="reason-line"> with innerHTML. No encoding or sanitization is applied.
Payload Used	<svg onload=document.title='XSS_CONFIRMED'></svg> (HTML-text payload)
Verification Method	verify_xss_execution – checked document.title (DOM title) → executed (title changed to XSS_CONFIRMED). Screenshot captured as proof.
Severity	High – arbitrary JavaScript execution on a corporate-controlled block page.
Impact	An attacker can run any JavaScript in the context of the corporate network, potentially stealing credentials, bypassing security controls, or performing actions on behalf of the user.
Proof Artifact	Screenshot (artifact ID: screenshot_artifact) showing page title XSS_CONFIRMED.
Finding ID	xss_ed2562c7 (saved via save_xss_finding).

5. Сохранена находка (save_xss_finding):

- Тип уязвимости: reflected
- URL: [REDACTED]
- Источник: param: state
- Контекст: redirect_url (instanceUrl) via location.replace
- Payload: javascript:document.title='XSS_CONFIRMED'//
- Proof: page-tab_1.png (скриншот с найденным маркером)

Two sides of the coin

1

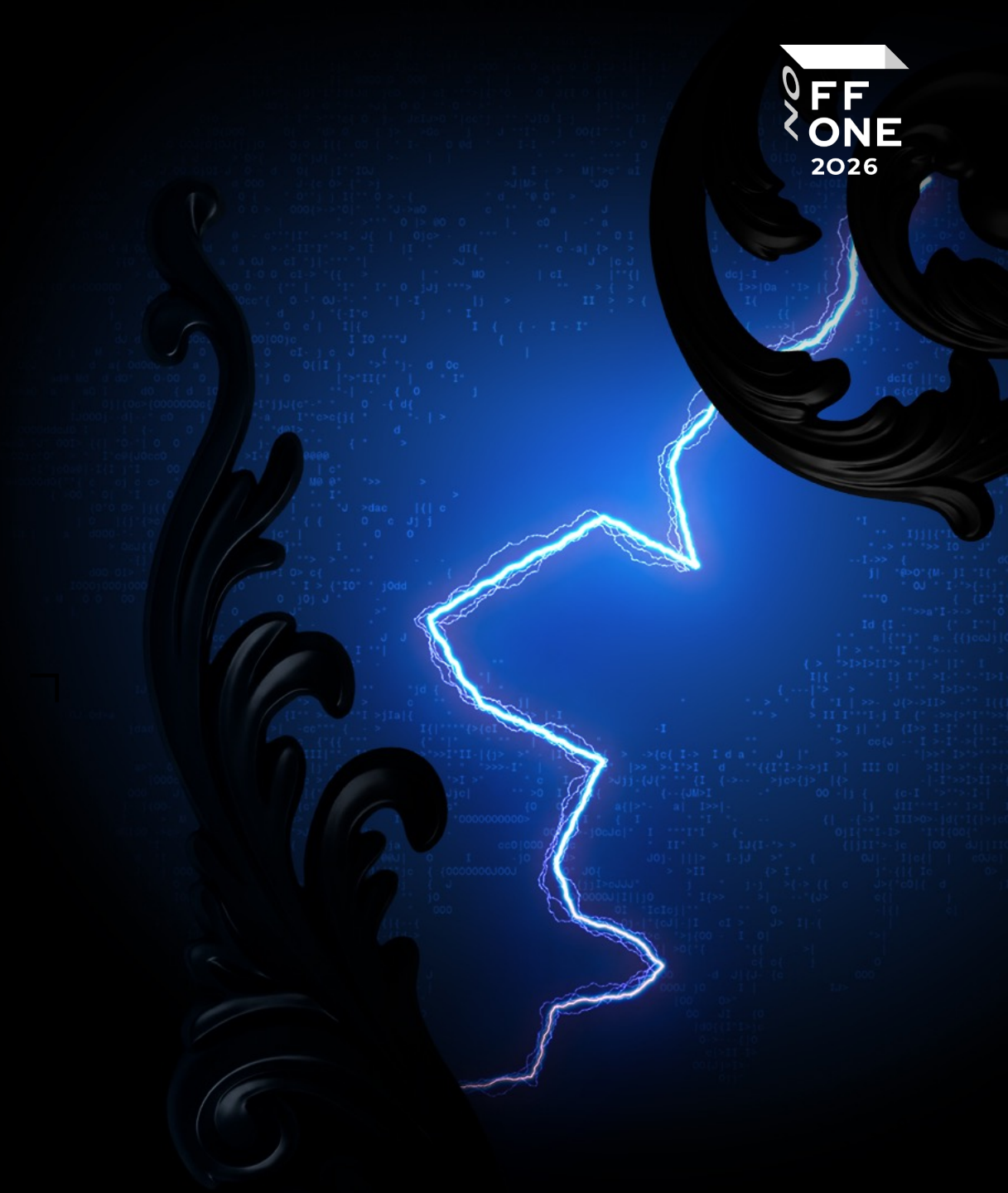
Determinism

Destroys LLM creativity

2

Excessive freedom

For an LLM freedom = a chance to “leave early”



Moving to a different architecture

- Everything changed fast, and by the end of 2025:
 - Models got significantly “smarter”
 - Skills appeared (<https://agentskills.io/home>)
 - More materials and research kept coming out
 - MCP, community-written skills, others’ experience
 - <https://github.com/SimoneAvogadro/android-reverse-engineering-skill>
 - <https://github.com/elementalsouls/Claude-BugHunter>
 - <https://github.com/dreadnode/capabilities>

What seemed sufficient

01

Browser and console

02

A powerful model and skills

03

Memory and state

Problems

Problem #1: Opaque execution

- Too many traces — little visibility into a run's progress after several hours

Problem #2: Achieving deep and broad coverage at the same time

- Fan out $\neq$ deep hunt
- “Find all the bugs, don't make mistakes” - does not work (*unfortunately*)

Problem #3: Run duration

- Agents are lazy and hallucinate
- Agents want to finish and exit as soon as possible

What turned out to be necessary

Skills

Technical reinforcement for the agent during testing

State

Describes the state of the current run per target

Memory

Helps to stay oriented in context and not spend tokens on work already done

Tools

Browser, HTTP-tool, Bash, OOB-kit, MCP, APK

Auth

OTP / SMS / Email gateways, infrastructure with live sessions

Observability

Hypotheses, traffic distributed across workers, scope control

State

Structured key-value with an explicit schema:

- Workers
- Endpoints and coverage
- Hypotheses
- Target model
- Finding lifecycle

https://example.com

COST

<mask>

TOTAL TOKENS

<mask>

in <mask> · out
<mask>

DURATION

999h

2000-01-01
00:00:00

TERMINATION

running

26 worker rounds

MODEL

<model>

worker <model>

— HYPOTHESES

X CONFIRMED

Y TESTED

Z QUEUED

N BLOCKED

M REJECTED

— TASKS

X RUNNING

Y QUEUED

Z BLOCKED-ON-DEPS

N COMPLETED

M DONE

K FAILED

— ENDPOINT COVERAGE 327 ENDPOINTS · 94 OBJECTS · 237 INVARIANTS

METHOD	HOST / PATH	STATUS	AUTH	SOURCE
POST	/v3/links.content		guest+csrf-gated	page.js getSharedThreadUnauthed
POST	/v3/links.comment		authenticated	page.js postCommentWithIDToken
POST	/v3/links.commentToken		public	page.js
POST	/v3/links.open		authenticated	inventory+live
POST	/v3/links.joinTeam		authenticated	page.js
POST	/v3/links.joinTeamToken		public	page.js
GET	/v3/csrfToken.create		public	page.js _csrfToken
GET	/ping		public	fuzz

— TARGET MODEL

- Roles (9)
- Capabilities (15)
- Attack chains (1 · 4 nodes · 2 edges)

- Practices already described: <https://agentskills.io/skill-creation/best-practices>
- Bigger size $\neq$ better skill quality
- Almost no ready plug-and-play skills exist for your tasks
 - Methodology, not a checklist
 - How to verify and how to behave — not a list of payloads
 - Always needs adaptation to your own harness
- Models are getting “smarter”:
 - less “technical” description of common vulnerabilities, more coverage methodologies and completeness recommendations
 - consolidating skills matters more than their count in the system

Observability

- Custom traffic attribution per agent
 - Storage of all sent requests and responses with HTTPQL support, evidence of checks and vulnerability exploitation
- ScopeJudgement — an external LLM controls the “safety” of actions
- Hypothesis lifecycle
- Trace and metric collection

16:55:26 STATE.QUEUE_HYPOTHESIS · COMPLETED

```
{ "claim": "shared-thread HTML sanitizer (DOMPurify with ALLOW_UNKNOWN_PROTOCOLS:true + custom afterSanitizeAttributes hook _fixSourceUrls/_makeAllLinksOpenInNewTab that rewrite src/srcset/href/cid POST-sanitization) can be bypassed to achieve DOM XSS when a crafted email/shared-thread body is rendered via dangerouslySetInnerHTML. leading to account
```

19:27:26 STATE.MARK_HYPOTHESIS · COMPLETED

```
{ "hypothesis_id": " ", "status": "rejected", "inline_evidence": " "
```

```
base also returns count:0 (own data only). Ownership scoping holds; not an IDOR.", "reason": "param does not override JWT tenant filter"}
```

HYPOTHESIS DETAIL

confirmed

graphql_https_ GraphQL_mutations_on_ _allow_anonymo · graphql · medium

GraphQL mutations on allow anonymous or authenticated data modification, or GraphQL query batching/aliasing enables authz bypass or rate-limit bypass

WORKED BY: task_default_001 · default bug-graphql

Workers

- Role segmentation
- Different sets of tools/skills
- Model selection per target

```
@enqueue_worker(...,  
    goal="",  
    task=[],  
    skills=[],  
    hyphoteses=[],  
    ...)
```

```
19:18:13 STATE.ENQUEUE_WORKER · COMPLETED  
  
type=default skills=['bug-mass-assignment']  
Test registration mass-assignment on (hvd-register-massassign-003).  
  
open worker task_default_005 → go to trace ↗
```

task_idor_028 · idor

← run trace →

STATUS	FINDINGS	DURATION	SCHEMA
completed	1 3 leads · 14 dead-ends	30m17s exit 0	valid 5 missing paths

Memory

- Event-sourced memory
- Short-term — progress within the current run
- Long-term — across runs
 - Indexing for easy search and deduplication
 - Discovered vulnerabilities, covered hypotheses
 - General knowledge about the application
 - Related applications (including mobile)
- Knowledge Base
 - Skills, methodology
 - CVEs for the application's stack

01 Recon

Active - dedicated workers:

- JS, .APK

Passive:

- HTTP-traffic

02 Browser

- Antibot
- DOM manipulation,
not clicking screenshots

03 MCP

- OOB interactions,
tools, ...

- ```
03:01:19 STATE.RECORD_SESSION - COMPLETED
```

## — AUTH SESSIONS (8)

20

# What keeps the agent going?

## 01 State

The agent doesn't know its exit conditions

## 02 Workers

The agent doesn't pull workers — their work progress is injected into the conversation

## 03 Prompt hygiene

The agent must not know it's part of a measured system  
*otherwise it will look for exit routes to finish the job faster*

# What else not to miss?

## 01 Token economy

- The harness must spend a minimum of tokens — the maximum goes to exploring the app and verifying vulnerabilities
- Token caching

## 02 Scope enforcement

- **Problem:** the agent can drift to neighboring or out-of-scope applications
- **Solution:** a dedicated control solution (LLM + network proxy)

## 03 API matrix

- Coverage completeness
- No testing of objects you don't own

# Verification



# Verification

- A separate verification process by critic workers (not part of any other orchestration)
- A false positive wastes operator time; a false negative will be caught in later runs
- Every finding is the start of a chain, not an agent's "victory"
- "You are a program triager. The bug hunter is trying to fool you..."

**pending\_critic** · race · medium · authenticated

Inventory double-booking

FOUND BY: **orchestrator** · CRITIC: **task\_finding-critic\_010**

**discard** · **critic: discard** · other · high · unauthenticated

Open redirect via client-side ref

FOUND BY: **task\_idor\_018** · idor · bug-open-redirect · CRITIC: **task\_finding-critic\_014**

**submit** · ssrf · high · unauthenticated

Full-read SSRF REST proxy via returns  
response body inline, no IP filtering →

Full-read server-side proxy — cloud metadata credential theft, k8s API access, internal network scanning with full response body return. Most severe SSRF variant.

FOUND BY: **orchestrator**

# Model differences

- Different models find DIFFERENT bugs
- Same target, same conditions — different findings:

**GLM 5.2**

3 crit / 1 high / 5 medium / 2 low

**Opus 4.8**

1 crit / 1 high / 3 medium / 5 low

# Model differences (by vulnerability type)

| Vulnerability type     | Model, % of findings (~) |         |      |     |
|------------------------|--------------------------|---------|------|-----|
|                        | Qwen 3.6                 | GLM 5.2 | Opus | GPT |
| IDOR / Auth            | 35                       | 50      | 45   | 30  |
| Business Logic         | 10                       | 20      | 25   | 30  |
| SSRF                   | 10                       | 10      | 10   | 5   |
| Client-Side            | 25                       | 10      | 15   | 20  |
| Info disclosure, other | 20                       | 10      | 5    | 15  |

# Model differences (on a lab sample)

01

Testing small models with 200k context on labs:

- Qwen3.6-35B - 73% pass rate
- Sonnet 4.8 - 80% pass rate

02

Testing small models on live targets:

- Qwen3.6-35B - 1-2 findings per run
- Sonnet 4.8 - 5-6 findings

03

Other tested models:

- Qwen 3.5-27B, gpt-120b-oss
- <https://huggingface.co/Jackrong>
- Qwopus3.6-27B-v2-FP8
- Qwen3.5-27B-v3
- Qwen3.5-27B-Claude-4.6-Opus-Reasoning-Distilled

# Results so far



**1400+**

findings submitted by the agent  
First round of “AI slop”

**600+**

findings approved by the critic  
agent before deduplication  
Second round of “AI slop”

**200+**

operator-confirmed vulnerabilities (~):  
16% critical, 30% high  
27% medium, 17% low, 10% info

# Benchmarks

## How we benchmarked

- Before - “WOW my agent solved 90% of XBOW labs!!! 🤪🤪🤪”
- Now - “where to find labs which are not in training dataset...”

## Why still benchmark

- Token savings before real runs
- A way to try new models and hypotheses

## How to test your own work now?

- Write your own labs and a harness around them
- Look for freshly released labs/tasks/machines

# Development challenges

- The bottleneck is testing
  - “Bad” hypotheses still burn tokens
    - Confirming or refuting a hypothesis takes several runs

# Model training

## Hypothesis

Train Qwen 3.6 on Opus traces

## Result

- Vanilla Qwen: 73% solve rate on the lab sample and 30-40 steps
- Finetuned Qwen: 41% and 100-120 steps

## Problems

The model inherited the reasoning and mechanics, but not the ability to solve

## Work in progress 😊

# What drives development now

“No need to be a Mythos to do offensive security” by Patrick Ventuzelo @ Le Hack 2026

“If a large model finds a 0-day with 90% probability, and a small model with 50% probability, but the small model costs 10x less, it is better to use the small model.” by live overflow @ [hacktron.ai](https://hacktron.ai)

<https://www.hacktron.ai/blog/why-mythos-doesnt-matter-for-us>

[https://fuzzinglabs.com/wp-content/uploads/2026/06/Le-Hack-2026-Keynote\\_-\\_No-Need-to-be-a-Mythos-to-do-Offensive-security-Patrick-Ventuzelo\\_-\\_FuzzingLabs.pdf](https://fuzzinglabs.com/wp-content/uploads/2026/06/Le-Hack-2026-Keynote_-_No-Need-to-be-a-Mythos-to-do-Offensive-security-Patrick-Ventuzelo_-_FuzzingLabs.pdf)

OFFZONE  
2026

Thank you!  
Questions?



OFFZONE  
2026

CICADA<sup>8</sup>



@CICADA8\_RU

